

# Tutoriel – GitHub Projects et travail collaboratif

---

## Objectif

Dans ce tutoriel, vous allez préparer l'environnement de travail de votre groupe pour le mini-projet.

Vous allez apprendre à :

- créer un dépôt GitHub ;
- ajouter les membres de votre groupe comme collaborateurs ;
- créer un **GitHub Project** ;
- associer le Project au dépôt ;
- ajouter les membres du groupe au Project ;
- créer des tâches sous forme d'issues ;
- organiser les tâches dans un tableau ;
- vérifier que tous les membres peuvent travailler sur le projet.

**Organisation** : groupes de 2 à 3 étudiants.

Un étudiant du groupe crée le dépôt et le Project. Il devient le responsable initial du dépôt. Il invite ensuite les autres membres du groupe.

---

## 1. Créer le dépôt GitHub

Un **repository** (dépôt) GitHub permet de stocker le projet et son historique Git.

Connectez-vous à votre compte GitHub.

Rendez-vous sur :

GitHub

Cliquez sur **New repository**.

Choisissez un nom pour votre projet.

Par exemple :

```
gestion-materiel-groupe1
```

ou :

```
suivi-incidents-groupe2
```

## Paramètres

Pour ce projet, utilisez les paramètres suivants :

- **Repository name** : nom fourni ;
- **Visibility** : Public ;
- ne créez pas automatiquement de README ;
- ne créez pas automatiquement de `.gitignore` ;
- ne choisissez pas de licence.

Cliquez sur **Create repository**.

Le dépôt est maintenant créé.

---

## 2. Ajouter les membres du groupe au dépôt

Le dépôt appartient pour le moment à l'étudiant qui l'a créé.

Les autres membres doivent être ajoutés comme **collaborateurs** afin de pouvoir travailler sur le dépôt.

GitHub permet au propriétaire d'un dépôt personnel d'inviter des collaborateurs depuis les paramètres du dépôt. Une fois l'invitation acceptée, les collaborateurs peuvent notamment récupérer et envoyer des modifications au dépôt.

### Étape 1 – Ouvrir les paramètres du dépôt

Dans votre dépôt :

**Settings → Collaborators**

Selon l'interface affichée, vous pouvez également trouver cette rubrique sous :

**Settings → Access → Collaborators**

### Étape 2 – Ajouter un étudiant

Cliquez sur :

**Add people**

Recherchez le **nom d'utilisateur GitHub** de votre camarade.

Sélectionnez son compte puis confirmez avec :

**Add NAME to REPOSITORY**

GitHub envoie alors une invitation à l'étudiant. GitHub Docs

### Étape 3 – Accepter l'invitation

Chaque étudiant invité doit accepter l'invitation.

Une fois l'invitation acceptée, il dispose des droits nécessaires pour participer au travail sur le dépôt.

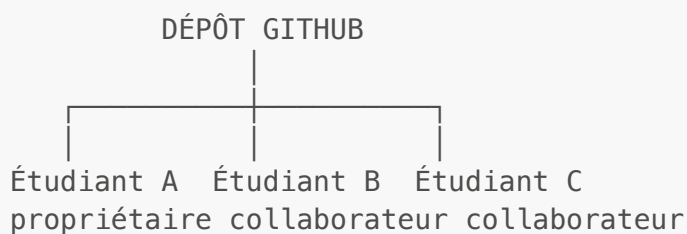
### Vérification

Dans :

### Settings → Collaborators

vous devez retrouver tous les membres du groupe.

Votre groupe est maintenant constitué de :



Pour un groupe de deux étudiants, il n'y aura naturellement que deux personnes.

---

## 3. Créer un GitHub Project

Un **GitHub Project** permet d'organiser et de suivre le travail du groupe.

Il peut notamment être utilisé sous la forme d'un tableau contenant différentes colonnes :

| À faire                       | En cours | Terminé |
|-------------------------------|----------|---------|
| Tâche 1<br>Tâche 4<br>Tâche 5 | Tâche 3  | Tâche 2 |

### Étape 1 – Accéder à Projects

Depuis GitHub, ouvrez la rubrique :

#### Projects

Puis choisissez :

#### New project

Choisissez une vue de type **Board**.

Donnez un nom au projet.

Par exemple :

Projet – Gestion de matériel

ou :

Projet – Suivi des incidents

---

## 4. Associer le Project au dépôt

Il est préférable d'associer votre Project au dépôt GitHub du groupe.

Depuis le Project, ouvrez :

### Settings

Puis recherchez le paramètre permettant de définir le **Default repository**.

Sélectionnez le dépôt créé précédemment.

Cette association permet notamment de faciliter la création d'issues directement liées au dépôt. [GitHub Docs](#)

---

## 5. Ajouter les membres du groupe au Project

L'accès au dépôt et l'accès au Project sont deux notions différentes.

Un étudiant peut avoir accès au dépôt sans nécessairement disposer des mêmes droits sur un Project.

Pour gérer les personnes ayant accès au Project :

### Project → Settings → Manage access

Dans la rubrique **Invite collaborators**, recherchez les membres du groupe.

Pour le mini-projet, donnez aux membres du groupe le rôle :

### Write

Ce rôle leur permet de consulter et de modifier le Project. [GitHub Docs](#)

---

## 6. Créer les premières tâches

Une tâche du projet peut être représentée par une **Issue**.

Une issue décrit un travail à réaliser.

Exemple :

Créer la page d'accueil

Une issue peut contenir :

- un titre ;
- une description ;
- une personne responsable ;
- une priorité ;
- une échéance ;
- des commentaires ;
- éventuellement des liens vers des Pull Requests.

---

## Exemple

Créez une première issue :

**Titre :**

Créer la page d'accueil

**Description :**

Créer la page d'accueil de l'application Flask.

La page doit présenter :

- le nom de l'application ;
- une courte description ;
- les principales fonctionnalités ;
- un lien vers la liste des données.

Attribuez cette tâche à un membre du groupe.

---

## 7. Créer le backlog du projet

Avant de commencer le développement, votre groupe doit identifier les principales tâches nécessaires à la réalisation du projet.

Par exemple :

Analyse du sujet  
Préparation du projet Flask  
Création de la page d'accueil  
Affichage des données

Création de la page de détail  
Ajout de la recherche  
Mise en forme CSS  
Tests  
Documentation  
Préparation de la livraison

Chaque tâche importante doit être transformée en **Issue**.

Vous devez ensuite retrouver ces tâches dans votre GitHub Project.

---

## 8. Organiser le tableau

Utilisez au minimum les trois états suivants :

À faire  
En cours  
Terminé

Au début du projet, toutes les tâches doivent se trouver dans :

### À faire

Lorsqu'un étudiant commence une tâche, elle passe dans :

### En cours

Lorsqu'elle est terminée et vérifiée :

### Terminé

Exemple :

| À faire                        | En cours                 | Terminé                         |
|--------------------------------|--------------------------|---------------------------------|
| Ajouter CSS<br>Tests<br>README | Affichage<br>des données | Structure Flask<br>Page accueil |

---

## 9. Attribuer les tâches

Chaque tâche doit être attribuée à un membre du groupe.

Exemple pour trois étudiants :

```
Étudiant A
→ Structure Flask
→ Page d'accueil
→ Tests

Étudiant B
→ Affichage des données
→ Page de détail

Étudiant C
→ CSS
→ README
→ Documentation
```

Cette répartition est seulement un exemple.

Vous devez choisir une organisation adaptée à votre projet.

**Important :** la répartition doit rester équilibrée. Chaque étudiant doit participer au développement et utiliser Git.

## 10. Créer votre Gantt prévisionnel

Avant de commencer réellement le développement, votre groupe doit réaliser un **diagramme de Gantt**.

Le Gantt doit représenter votre organisation prévisionnelle du projet.

Exemple :

| Tâche                 | S1 | S2 | S3 | S4 |
|-----------------------|----|----|----|----|
| Analyse               | ■  |    |    |    |
| Structure Flask       | ■  |    |    |    |
| Page d'accueil        |    | ■  |    |    |
| Affichage des données |    | ■  | ■  |    |
| CSS                   |    |    | ■  |    |
| Tests                 |    |    | ■  | ■  |
| README                |    |    |    | ■  |
| Livraison             |    |    |    | ■  |

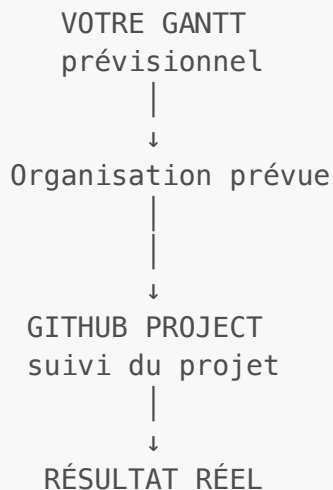
Le Gantt doit être réalisé **avant le développement**.

Il constitue votre **planning prévisionnel**.

## 11. Comparer votre Gantt avec GitHub Projects

GitHub Projects permet également d'organiser et de visualiser la planification du projet.

Votre objectif est de comparer :



À la fin du projet, vous devrez analyser les différences.

Par exemple :

| Tâche             | Prévu | Réel | Écart      |
|-------------------|-------|------|------------|
| Page d'accueil    | S1    | S1   | Aucun      |
| Affichage données | S2    | S3   | +1 semaine |
| CSS               | S3    | S2   | -1 semaine |
| Tests             | S3    | S4   | +1 semaine |

Vous devrez expliquer les principaux écarts.

Exemples :

- estimation trop optimiste ;
- difficulté technique ;
- tâche dépendante d'une autre tâche ;
- problème de compréhension ;
- absence d'un membre du groupe ;
- modification du projet ;
- tâche finalement plus simple que prévu.

---

## 12. Vérifier que le travail collaboratif fonctionne

Avant de commencer le projet, chaque membre du groupe doit effectuer une petite opération.



## Étudiant A

Clonez le dépôt :

```
git clone URL_DU_DEPOT
```

Puis placez-vous dans le projet :

```
cd nom-du-projet
```

## Étudiant B

Effectuez également un `git clone`.

Chaque étudiant doit disposer d'une copie locale du projet.

---

## Test de collaboration

Chaque étudiant doit ensuite :

1. créer ou modifier un petit fichier ;
2. effectuer un `git status` ;
3. utiliser `git add` ;
4. créer un commit ;
5. utiliser `git push` ;
6. vérifier que la modification apparaît sur GitHub.

Exemple :

```
git status
git add .
git commit -m "Ajout de ma contribution"
git push
```

L'objectif est de vérifier que **tous les membres du groupe disposent bien des droits nécessaires**.

---

## 13. Organisation du travail pendant le projet

Pour chaque nouvelle fonctionnalité, utilisez le workflow suivant :

```
ISSUE
  ↓
Attribution
```



Ce workflow doit être utilisé autant que possible pendant le projet.

---

## 14. Règles de travail du groupe

Votre groupe doit respecter les règles suivantes :

- chaque étudiant doit participer au développement ;
- chaque étudiant doit réaliser plusieurs commits ;
- les commits doivent avoir des messages explicites ;
- les tâches doivent être suivies dans GitHub Projects ;
- les tâches doivent être attribuées à un membre du groupe ;
- le travail doit être réparti entre les membres ;
- les modifications doivent être régulièrement synchronisées avec GitHub ;
- les Pull Requests doivent être relues par un autre membre lorsque cela est demandé ;
- le dépôt `main` doit toujours contenir une version fonctionnelle du projet.

---

[https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/repository-access-and-collaboration/inviting-collaborators-to-a-personal-repository?trk=public\\_post\\_comment-text&utm\\_source=chatgpt.com](https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/repository-access-and-collaboration/inviting-collaborators-to-a-personal-repository?trk=public_post_comment-text&utm_source=chatgpt.com)

[https://docs.github.com/en/issues/planning-and-tracking-with-projects/managing-your-project/managing-access-to-your-projects?utm\\_source=chatgpt.com](https://docs.github.com/en/issues/planning-and-tracking-with-projects/managing-your-project/managing-access-to-your-projects?utm_source=chatgpt.com)

---